

Nama: A. Issadurrofiq Jaya Utama

SurabayaDev Developer Technical Assessment

Assessment ini bertujuan untuk mengetahui kemampuan kandidat dalam memahami masalah teknis, menentukan pendekatan frontend/backend, membuat implementasi, serta menunjukkan kualitas engineering melalui repository project.

PART 1: TECHNICAL CASE STUDY

Case Study:

SurabayaDev membutuhkan sistem registrasi event dengan fitur:

- user melihat daftar event;
- user melakukan pendaftaran;
- user mendapatkan tiket digital;
- panitia melakukan validasi tiket saat acara;
- admin dapat melihat data peserta.

Kondisi sistem:

- 3.000 user mengakses website;
- 1.000 user melakukan registrasi;
- setiap event memiliki kuota tiket yang harus dikelola oleh sistem.

Pertanyaan

1. System Analysis

Jelaskan fitur yang dibutuhkan, flow pengguna, dan pembagian tanggung jawab frontend serta backend.

Jawaban:

a. Fitur yang Dibutuhkan:

- **Katalog & Pencarian Event:** menampilkan kartu event publik dengan pencarian realtime, filter kategori, status kuota, dan jadwal.
- **Otentikasi & Registrasi User:** pendaftaran akun aman (bcrypt cost 12), proteksi session (NextAuth v5), serta pemisahan role (User Peserta vs Admin Panitia).
- **E-Ticketing Digital:** penerbitan tiket unik otomatis dengan format SBYDEV-{6 karakter ID event}-{8 karakter kode acak} (contoh: SBYDEV-ABC123-XY789ZWQ), dilengkapi kode QR dinamis.
- **Workstation Scanner Check-In:** scanner berbasis pilihan event (2-step validation) untuk validasi tiket peserta di venue acara dan pencegahan salah event / duplicate check-in.
- **Dashboard Admin (CRUD & Data Peserta):** manajemen pembuatan event, monitoring pendaftar, dan rekap kehadiran.

b. Flow Pengguna (User & Admin Flow):

1. User Flow: Mengakses Katalog Event → Membuka Detail Event → Menekan Daftar (sistem memeriksa kuota via database transaction) → Menerima Tiket Digital & QR Code di menu 'Tiket Saya'.
 2. Admin / Panitia Flow: Login Admin → Membuka Workstation Scanner (/admin/scanner) → Memilih event yang aktif di venue → Memindai / memasukkan kode tiket peserta → Sistem memverifikasi kecocokan event & mencatat timestamp kehadiran secara real-time.
- c. **Pembagian Tanggung Jawab Frontend & Backend:**
- **Frontend (Client Layer):** bertanggung jawab atas rendering UI responsif, state management interaktif, instant client-side validation (Zod schema), rendering dynamic QR code, serta optimistic UI update.
 - **Backend (Server & Database Layer):** bertanggung jawab atas business logic enforcement, atomic transaction untuk kuota, validasi otentikasi & role authorization (RBAC), password encryption (bcrypt cost 12), serta verifikasi keaslian tiket.

2. Frontend Perspective

Jika kamu menjadi frontend developer, jelaskan teknologi yang digunakan, struktur component, state management, dan strategi menjaga performance.

Jawaban:

- a. **Teknologi yang Digunakan:**
- **Framework:** Next.js 15 (App Router) + React 19 + TypeScript untuk type-safety.
 - **Styling & Design System:** Tailwind CSS v4 dengan centralized CSS Variables sesuai Brand Identity SurabayaDev (Emerald Green #16a34a & Tech Slate #0f172a).
 - **Icon Library:** @phosphor-icons/react (Zero Emoji Policy).
 - **Animasi & Interaksi:** Motion (motion/react) untuk micro-interactions halus.
- b. **Struktur Component (Feature-Based Modular Architecture):**
- Menggunakan struktur modular per fitur (features/events, features/tickets, features/admin, features/auth) dan aturan mikro-komponen (idealnya <150 baris per file). Komponen repetitif diisolasi mandiri, seperti ProfileDropdown, LogoutModal, ScannerEventSelector, ScannerEventBanner, dan ScannerSessionLog.
- c. **State Management:**
- **Session State:** NextAuth SessionProvider membungkus seluruh aplikasi untuk menyediakan data autentikasi & role secara global.
 - **Server-Fetch State per Fitur:** setiap fitur memiliki custom hook mandiri (useEvents, useEventDetail, useEventRegistration, useTickets, useTicketDetail, useAdminEvents) yang mengelola data, loading, dan error state menggunakan useState/useEffect/useCallback lalu memanggil REST API secara langsung.
 - **Persiapan Skala Lanjutan:** Zustand dan TanStack Query sudah disiapkan sebagai dependency di package.json untuk pengembangan global state dan server-cache yang lebih kompleks pada iterasi berikutnya.
- d. **Strategi Menjaga Performance (Menghadapi 3.000 Pengunjung):**
- **Server Component by Default:** halaman utama dirender di sisi server (async server component) untuk First Contentful Paint yang cepat dan SEO optimal.

- **Pagination Terbatas:** endpoint `/api/events` dipanggil dengan limit 6 event per halaman sehingga payload awal tetap kecil saat diakses ribuan pengunjung bersamaan.
- **Image Optimization:** menggunakan `next/image` dengan format WebP/AVIF modern, responsive srcset, dan blur placeholder.
- **Suspense & Skeleton Loading:** transisi rendering asynchronous menggunakan skeleton pulse untuk meminimalkan layout shift.

3. Backend Perspective

Jika kamu menjadi backend developer, jelaskan desain database, API yang dibutuhkan, validasi tiket, pengelolaan kuota tiket, dan cara mencegah duplicate check-in.

Jawaban:

- Desain Database (PostgreSQL + Prisma ORM):**
 - **Model User:** id (cuid), name, email (unique), password (hashed), role (USER/ADMIN), createdAt.
 - **Model Event:** id (cuid), title, description, date, location, quota, registered (counter), category, imageUrl, createdAt.
 - **Model Ticket:** id (cuid), code (unique), userId (relasi User), eventId (relasi Event), status (PENDING/CHECKED_IN/CANCELLED), checkedAt, createdAt. Unique constraint pada `@@unique([userId, eventId])` untuk mencegah pendaftaran ganda per akun pada event yang sama.
- API Endpoints yang Dibutuhkan:**
 - POST `/api/auth/register` & `/api/auth/[...nextauth]` — registrasi & sesi otentikasi.
 - GET `/api/events` & POST `/api/events` — katalog publik (dengan filter pencarian/kategori) & pembuatan event (admin).
 - GET `/api/events/[id]`, PUT `/api/events/[id]`, DELETE `/api/events/[id]` — detail, update, dan hapus event (admin).
 - POST `/api/events/[id]/register` — registrasi tiket dengan quota transaction.
 - GET `/api/events/[id]/attendees` — daftar peserta per event (admin).
 - GET `/api/tickets` & GET `/api/tickets/[code]` — daftar tiket user & detail tiket.
 - POST `/api/tickets/[code]/checkin` — validasi & check-in oleh panitia (admin).
- Pengelolaan Kuota Tiket (Anti-Race Condition saat 1.000 Registrasi Serentak):**
Menggunakan database transaction (prisma.\$transaction) dengan atomic condition:
 1. Memeriksa ketersediaan kuota secara terisolasi (`registered < quota`).
 2. Memeriksa duplikasi pendaftaran user pada event tersebut.
 3. Membuat tiket baru dan melakukan atomic increment (`registered + 1`) dalam satu siklus commit. Jika kuota sudah penuh, transaksi dibatalkan dan sistem mengembalikan error `QUOTA_EXCEEDED`.
- Validasi Tiket & Pencegahan Duplicate Check-In:**
 - **Validasi Kesesuaian Event (Event-Mismatch Prevention):** endpoint check-in menerima parameter `eventId` dari scanner. Jika tiket terdaftar untuk event lain, sistem menolak dengan error code `EVENT_MISMATCH`.
 - **Pencegahan Duplicate Check-In:** transaksi memeriksa kolom status tiket. Jika status sudah `CHECKED_IN`, sistem menolak dengan HTTP 409 (`ALREADY_CHECKED_IN`).

4. Technical Decision

Jelaskan bagian tersulit dari sistem dan solusi teknis yang kamu pilih.

Jawaban:

Bagian Tersulit: menangani high-concurrency ticket war (ribuan user mendaftar bersamaan pada sisa kuota terbatas) dan menjamin integritas data check-in di lokasi acara.

Solusi Teknis yang Dipilih:

1. Database Atomic Transactions: menggunakan `prisma.$transaction` untuk mengeliminasi celah race condition tanpa membutuhkan message broker di tahap awal.
2. Single Source of Truth (SSOT) Architecture: repository pattern pada lib/db/ memisahkan business query dari routing layer. Tipe TypeScript diturunkan langsung dari Prisma Client untuk mencegah type drift.
3. High-Order Middleware Guards: `withErrorHandler` dan `requireAdmin()` membungkus seluruh endpoint untuk standardisasi format JSON { success, data/error }.
4. 2-Step Workstation UX: scanner panitia dikunci pada event aktif tertentu untuk mencegah kesalahan panitia saat memvalidasi tiket di tengah antrean padat.

PART 2: CODING CHALLENGE

Buat aplikasi sederhana sesuai fokus yang dipilih.

Peserta dapat memilih salah satu:

A. Frontend Developer

Membuat Event Dashboard dengan fitur:

- daftar event;
- pencarian event;
- detail event;
- tombol registrasi.

B. Backend Developer

Membuat Event Management API dengan fitur:

- CRUD event;
- registrasi user;
- database;
- validation;
- error handling;
- API documentation.

C. Full-stack Developer

Membuat Mini Event Platform dengan:

- event list;
- authentication;
- CRUD event;
- registration.

Jawaban:

Fokus yang Dipilih: C. Full-stack Developer (Mini Event Platform).

Fitur yang Telah Diimplementasikan:

- Event list, pencarian, dan filter kategori.
- Autentikasi (NextAuth v5) dengan role User & Admin.
- Admin Event CRUD lengkap dengan image banner dropzone.
- Registrasi tiket dengan atomic quota transaction.
- Tiket digital dengan QR code dinamis.
- Workstation scanner check-in 2 langkah untuk panitia.

Link GitHub Repository: <https://github.com/DevIssa-It/sbydev>

Link Live Demo: <https://sbydev.vercel.app>

Live demo menggunakan hosting apa pun menjadi nilai tambah.

Akun Demo Pengujian:

- Akun Admin: admin@sbydev.id | Password: admin123
- Akun User: user@sbydev.id | Password: user123

NB

Proses development melalui repository akan menjadi bagian dari penilaian. Reviewer akan melihat history commit, kualitas pesan commit, konsistensi perubahan kode, struktur project, serta dokumentasi yang dibuat. Gunakan commit yang jelas dan menggambarkan perubahan yang dilakukan selama pengerjaan.

PART 3: COMMUNITY COMMITMENT

Jawab pertanyaan berikut:

1. Mengapa kamu tertarik bergabung menjadi bagian dari komunitas SurabayaDev?

Jawaban:

Saya tertarik bergabung dengan SurabayaDev karena melihat rekam jejak dan dampak nyata komunitas ini dalam membangun ekosistem teknologi di Jawa Timur. SurabayaDev bukan sekadar wadah berkumpul, melainkan jembatan kolaborasi antara mahasiswa, praktisi, dan industri. Saya ingin berkontribusi langsung dengan keahlian software engineering yang saya miliki untuk memperkuat infrastruktur digital komunitas serta membagikan wawasan teknis kepada rekan-rekan developer lainnya.

2. Kontribusi apa yang ingin kamu berikan untuk perkembangan komunitas developer di Surabaya?

Jawaban:

Bentuk kontribusi yang ingin saya berikan meliputi:

- **Pengembangan & Pemeliharaan Platform Digital:** membantu membangun dan mengoptimalkan sistem web internal SurabayaDev (seperti platform ticketing, certificate generator, dan member hub).
- **Pemateri & Mentor Teknis:** mengisi sesi workshop, tech talk, atau hands-on lab mengenai topik web development modern, arsitektur cloud, dan clean code.

- **Pengorganisasian Event Teknologi:** turut aktif dalam kepanitiaan meetup rutin, hackathon tahunan, serta workshop intensif agar acara berjalan lancar dan berbobot.
3. Bagaimana cara kamu membagi waktu antara komunitas, kuliah/pekerjaan, dan aktivitas pribadi?

Jawaban:

Saya menerapkan manajemen waktu berbasis prioritas (Metode Eisenhower) dan time-blocking:

- Mengalokasikan blok waktu khusus mingguan untuk tugas komunitas secara terstruktur di luar jam kerja/kuliah utama.
 - Menggunakan tools produktivitas seperti Notion, Google Calendar, dan GitHub Project Board untuk memonitor milestone serta deliverable.
 - Menerapkan komunikasi proaktif dan transparan dengan tim mengenai kapasitas kerja agar setiap komitmen yang diambil dapat diselesaikan dengan kualitas terbaik tanpa mengorbankan tanggung jawab pribadi.
4. Apakah kamu bersedia tetap aktif berkontribusi setelah SurabayaDev 12th Anniversary selesai? Jelaskan bentuk kontribusi yang ingin dilakukan.

Jawaban:

Ya, saya bersedia dan berkomitmen untuk terus aktif berkontribusi dalam jangka panjang setelah rangkaian SurabayaDev 12th Anniversary selesai. Bentuk kontribusi jangka panjang yang ingin saya lakukan antara lain:

1. Menjaga kesinambungan platform teknologi SurabayaDev melalui pembaruan fitur berkelanjutan.
 2. Berperan aktif dalam regenerasi dan mentoring anggota baru komunitas.
 3. Membantu merancang kurikulum materi teknis untuk program meetup bulanan SurabayaDev berikutnya.
5. Ceritakan pengalaman ketika bekerja dalam tim komunitas, organisasi, atau project open source.

Jawaban:

Dalam pengalaman saya berkolaborasi di lingkungan tim teknis dan komunitas, saya terbiasa menerapkan standar rekayasa modern:

- Menggunakan Git Flow dengan Conventional Commits dan Pull Request review yang ketat untuk menjaga kualitas codebase.
- Membagi arsitektur proyek secara modular agar setiap anggota tim dapat bekerja paralel tanpa konflik merge.
- Menyelenggarakan stand-up meeting rutin dan dokumentasi teknis yang jelas (seperti panduan setup lokal dan kamus API) sehingga anggota tim baru dapat onboard dengan cepat.

Pengalaman ini melatih saya untuk tidak hanya fokus pada penulisan kode berkualitas, tetapi juga pada komunikasi yang efektif, empati antardivisi, dan orientasi pada penyelesaian masalah.